

Rung 3 — Composing error bounds (how small errors add up)

Where you start: you've bounded two error sources on their own — fixed-point truncation ($18/D$, Rung 1) and the Taylor remainder (x^6 , Rung 2). But a real computation stacks *many* approximate steps, and the errors interact. This rung is the rule for stacking them.

What you'll be able to do: take bounds on the pieces and produce a bound on the whole — and explain why the composed $\sin^2 + \cos^2$ certificate you ran on the FPGA is *looser* than the sin and cos certificates it's built from.

1. The bench

The `sincos_certificate_v0` kernel computes, on real silicon, $\sin_{\text{hw}}(x)^2 + \cos_{\text{hw}}(x)^2$ and checks it against the exact identity $\sin^2 + \cos^2 = 1$. Its margins, measured over 1200 samples:

```
sin certificate margin      ≈ 2.6e-4
cos certificate margin      ≈ 5.3e-4
composed (sin²+cos²) margin ≈ 1.1e-3    <- looser than either piece
```

Two questions: *why does combining two certified things give a **looser** bound?* And *how do you compute the combined bound from the pieces* — without just testing and hoping?

2. The one new idea: errors don't cancel, they accumulate — and you can bound the pile

Two tools cover almost everything.

(a) Adding approximate things — the triangle inequality. If $\hat{a}$ is off from a by at most E_a , and $\hat{b}$ off from b by at most E_b , then $\hat{a} + \hat{b}$ is off from $a + b$ by at most $E_a + E_b$:

$$|(\hat{a} + \hat{b}) - (a + b)| \leq |\hat{a} - a| + |\hat{b} - b| \leq E_a + E_b$$

That " $|X + Y| \leq |X| + |Y|$ " step is **the triangle inequality** — the single most-used move in all of analysis. Errors add.

(b) Multiplying approximate things — `qmul_err`. This is the subtle one. If $\hat{a} = a + e$ and $\hat{b} = b + f$ (with $|e| \leq E_a$, $|f| \leq E_b$), then

$$\hat{a} \cdot \hat{b} = (a+e)(b+f) = ab + a \cdot f + b \cdot e + e \cdot f$$

so the product's error is $|\hat{a} \cdot \hat{b} - ab| \leq |a| \cdot E_b + |b| \cdot E_a + E_a \cdot E_b$. In words: **each factor's error gets amplified by the other factor's size**, plus a tiny cross term. That's why an early rounding error doesn't stay small — it gets multiplied by everything downstream. (On real hardware there's also the `qmul` truncation itself, $\leq 1/D$, added on top — Rung 1.)

3. Do it by hand

The composed certificate squares each output and adds. Write $E_s = 18/D + x^6$ (the sin error, Rung 1+2) and $E_c = 36/D + x^7$ (the cos error — same shape, its own constants). Then:

- `sin_hw²` is `qmul(sin_hw, sin_hw)`. By the multiply rule with $a = b = \sin(x)$ and error E_s , plus the $1/D$ truncation: its distance from $\sin(x)^2$ is at most $1/D + (|\sin x| + E_s) \cdot E_s + E_s \cdot |\sin x|$.
- Same for `cos_hw²` with E_c .
- Add them (triangle inequality), and use the exact fact $\sin^2 + \cos^2 = 1$:

$$\begin{aligned} & |\sin_hw^2 + \cos_hw^2 - 1| \\ & \leq [1/D + (|\sin x| + E_s) \cdot E_s + E_s \cdot |\sin x|] + [1/D + (|\cos x| + E_c) \cdot E_c + E_c \cdot |\cos x|] \end{aligned}$$

That bracketed monster **is** the composed certificate's right-hand side — and now you see where every term comes from. Because each squaring *amplifies* the primitive error (the $|\sin x| \cdot E_s$ terms) and adds its own truncation, the total is bigger than E_s or E_c alone. **Composition loosens** — that's not a bug, it's arithmetic, and it's exactly the $1.1e-3 > 2.6e-4$ you measured.

4. Make it formal

`machlib` builds this the same way you just did — by *chaining* proven lemmas, not re-deriving from scratch:

```
FixedPoint.qmul_err          -- the multiply rule (a·f + b·e + e·f), proven once
MachLib.Trig.pythagorean    -- sin²+cos² = 1 (the exact identity)
MachLib.Real.sin_cos_sq_sum_fwd_error -- composes the two primitive certs via qmul_err + the identity
```

This is the real power of machine-checked proof: a certificate about a *system* is assembled by composing certificates about its *parts*, and Lean guarantees the assembly has no gaps. You proved the parts (Rungs 1–2); this composes them.

5. Watch it hold

On silicon: 1200 samples, **zero violations**, composed margin $1.12e-3$ — matching the by-hand bound — and `sin_hw² + cos_hw²` read exactly `0x10000 = 1.000000`. The composition you just did by hand is the thing the hardware confirmed.

The cliffhanger → Rung 4

That bracketed bound had maybe a dozen steps. The full machlib composition has *hundreds*. How do you know nobody dropped a sign or fumbled an inequality somewhere in the middle? You don't — so you hand the whole chain to a machine that mechanically checks every single step and refuses to accept a gap. That machine is **Lean**, and learning to drive it is the next rung.

→ **Next:** Rung 4 — *Make Lean check it* — propositions-as-types, and your first machine-checked proof.

Exercises & checkpoint

1. Let $\hat{a} = a + e$ and $\hat{b} = b + f$. Expand $\hat{a} \cdot \hat{b}$ and circle the three error terms. Which one is tiny, and why?
2. In one sentence using the $a \cdot f + b \cdot e + e \cdot f$ rule, explain why the composed $\sin^2 + \cos^2$ margin ($1.1e-3$) is *bigger* than the sin margin ($2.6e-4$).
3. Compute $E_s(0.5) = 18/D + 0.5^6$ and $E_c(0.5) = 36/D + 0.5^7$. Roughly how big is each? (You don't need the full composed formula — just feel the sizes.)

Checkpoint: you can explain, with the multiply-error rule, why composing certificates loosens the bound.